CURSO: PROGRAMACIÓN ORIENTADA A OBJETOS

Engine 2D

Arquitectura de Software, Relaciones entre Clases y Principios SOLID

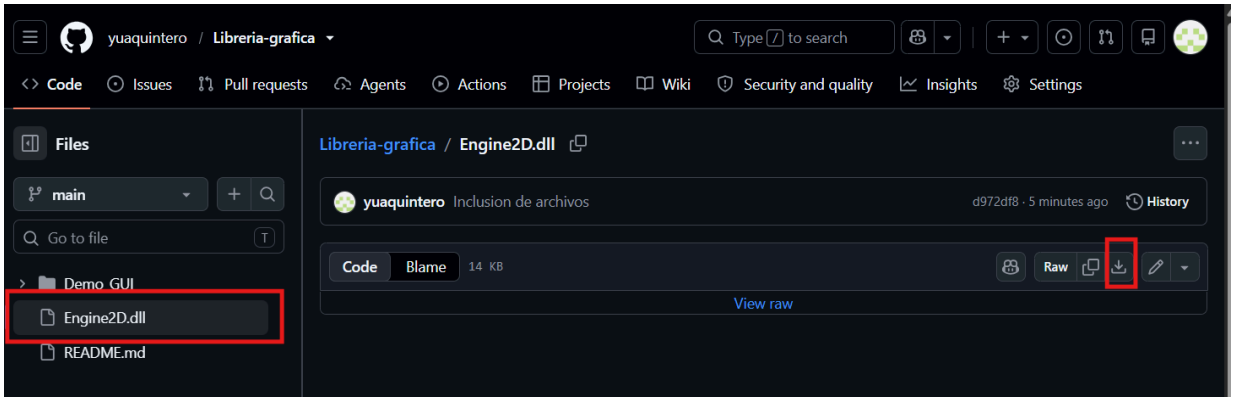
Indicadores de resultado de aprendizaje

- Representar las relaciones entre clases a través de diagramas UML en ejemplos de sistemas de información.
- Diseñar diagramas de clases en UML con apoyo de IA, representando correctamente las relaciones, atributos y métodos.

Paso 1: Descarga de la Librería

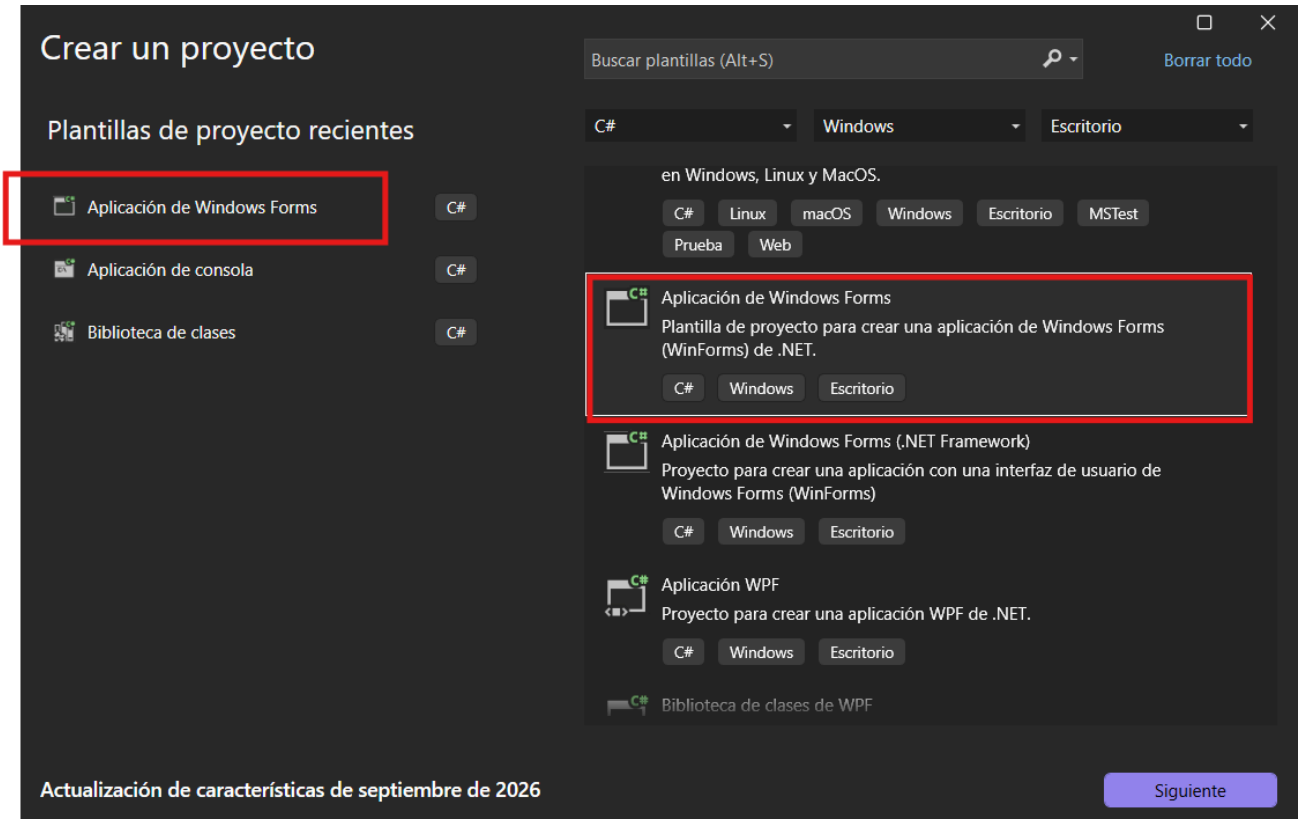
1. Accede al repositorio en GitHub de la asignatura.

   https://github.com/yuaquintero/Libreria-grafica/blob/main/Engine2D.dll
2. Ubica la carpeta del proyecto y selecciona el archivo binario **Engine2D.dll**.
3. Haz clic en el botón de descarga (**Download / Raw**) para guardar la librería en tu equipo.

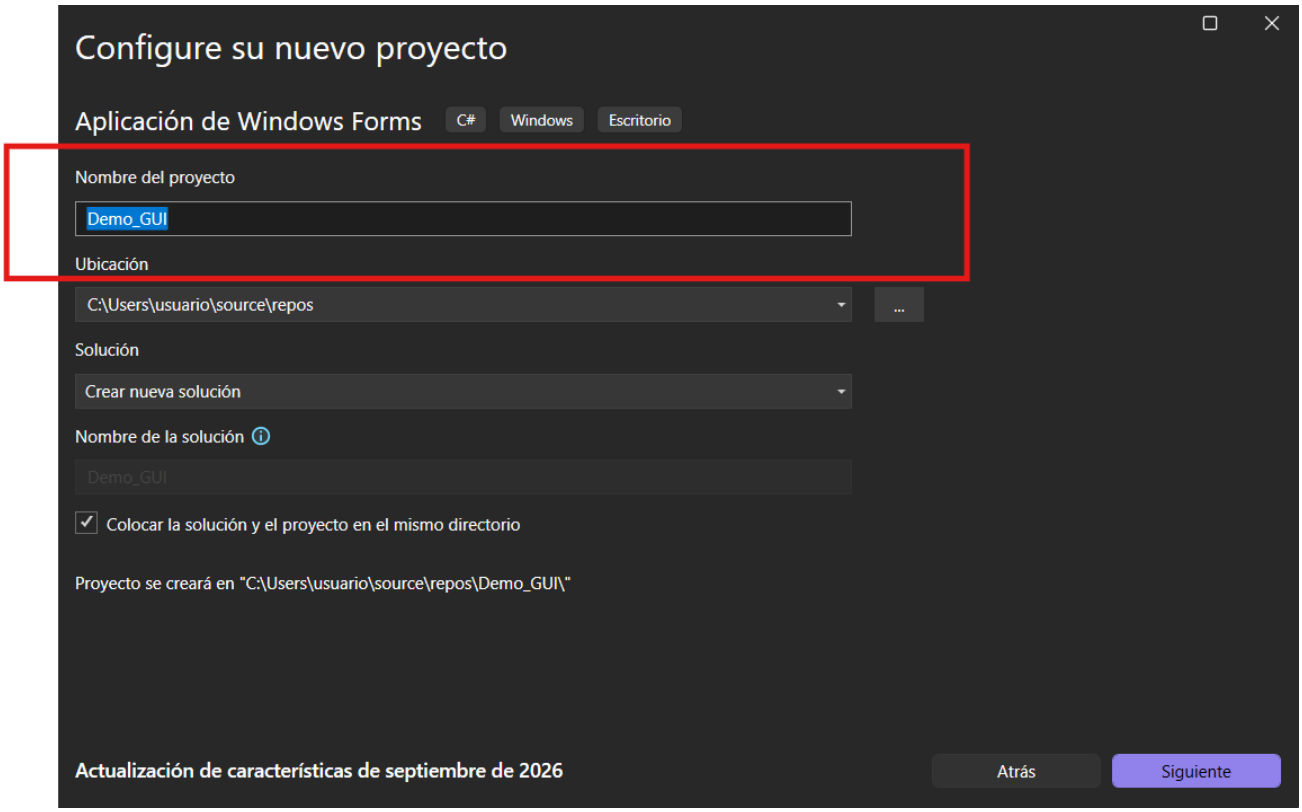


Paso 2: Creación del Proyecto en Visual Studio

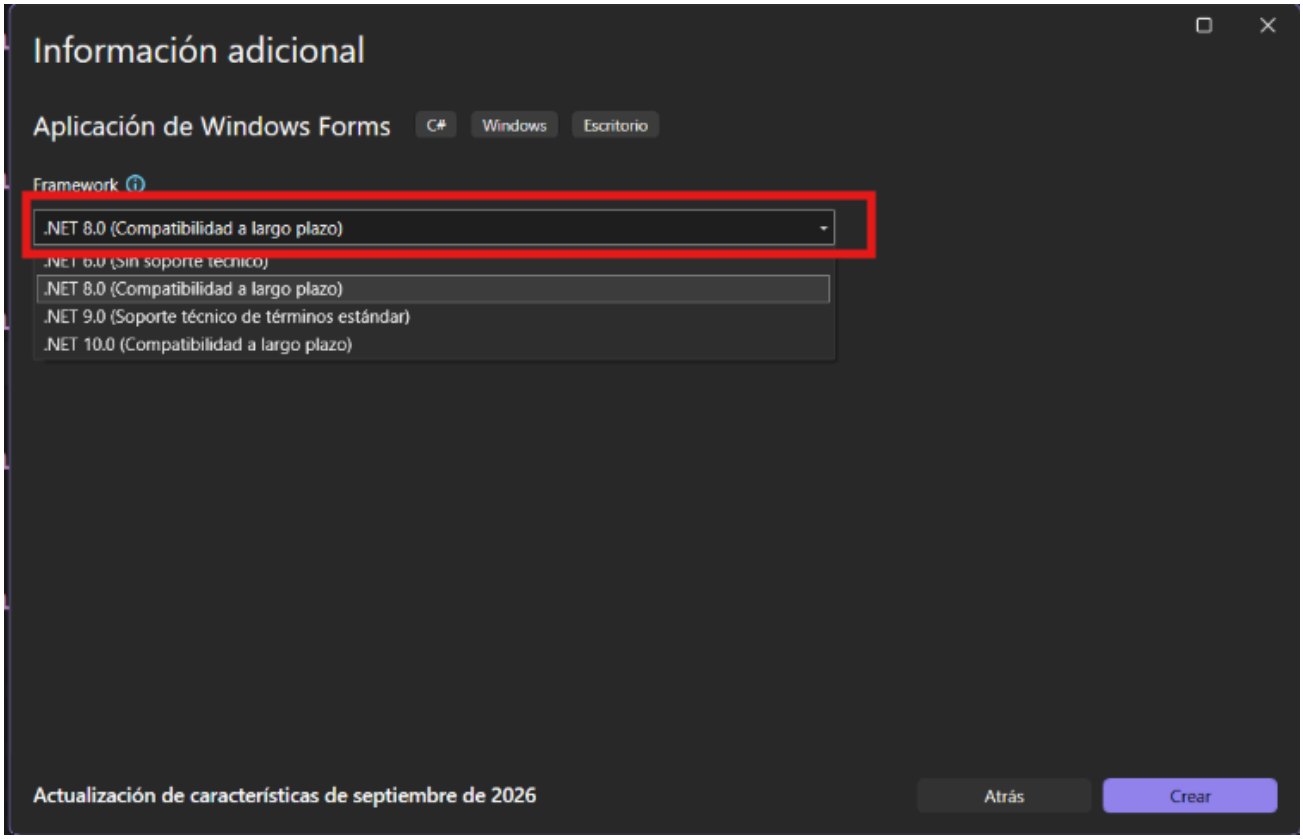
1. Abre Visual Studio y selecciona **Crear un proyecto**.
2. Filtra por plantilla y selecciona **Aplicación de Windows Forms** (asegúrate de que corresponda a C# para .NET).



3. Asigna un nombre al proyecto (por ejemplo, Demo_GUI), selecciona la ubicación de guardado y haz clic en **Siguiente**.

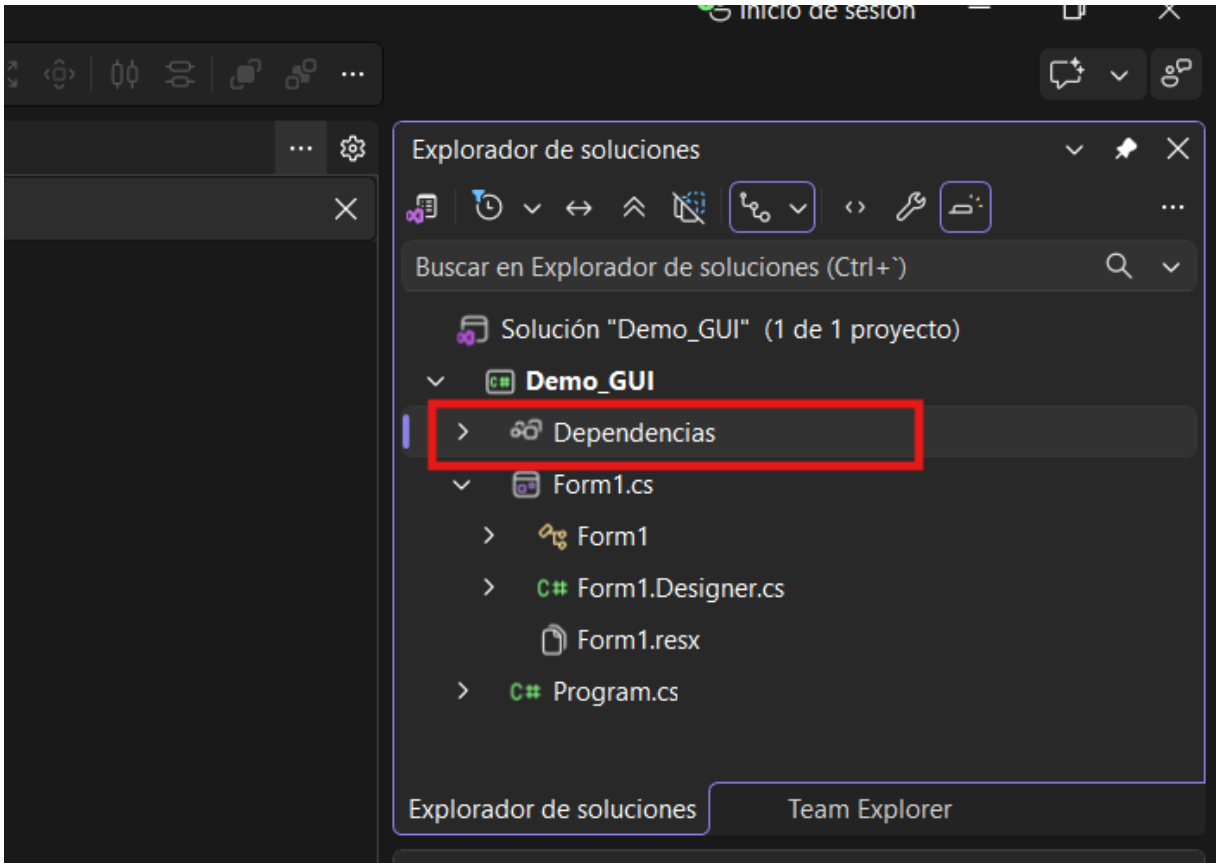


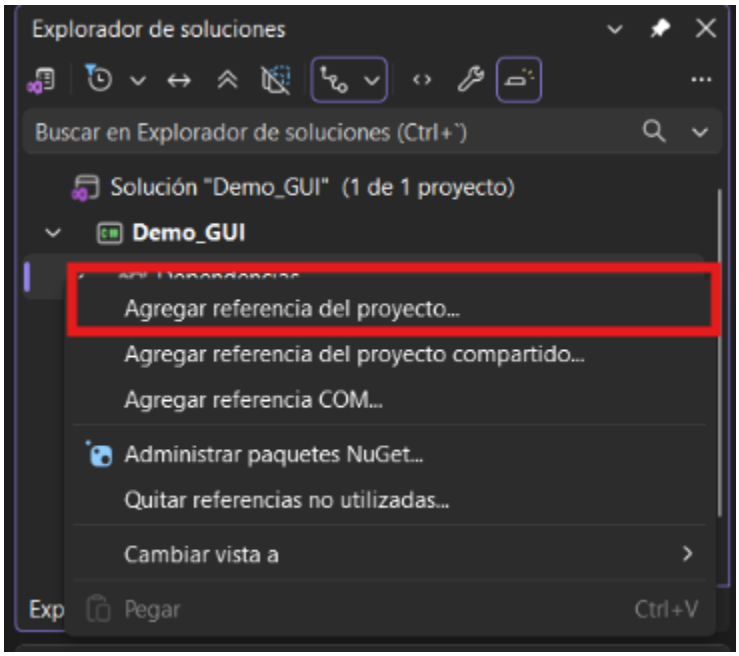
4. En el framework de destino, selecciona **.NET 8.0 (Compatibilidad a largo plazo)** y haz clic en **Crear**.



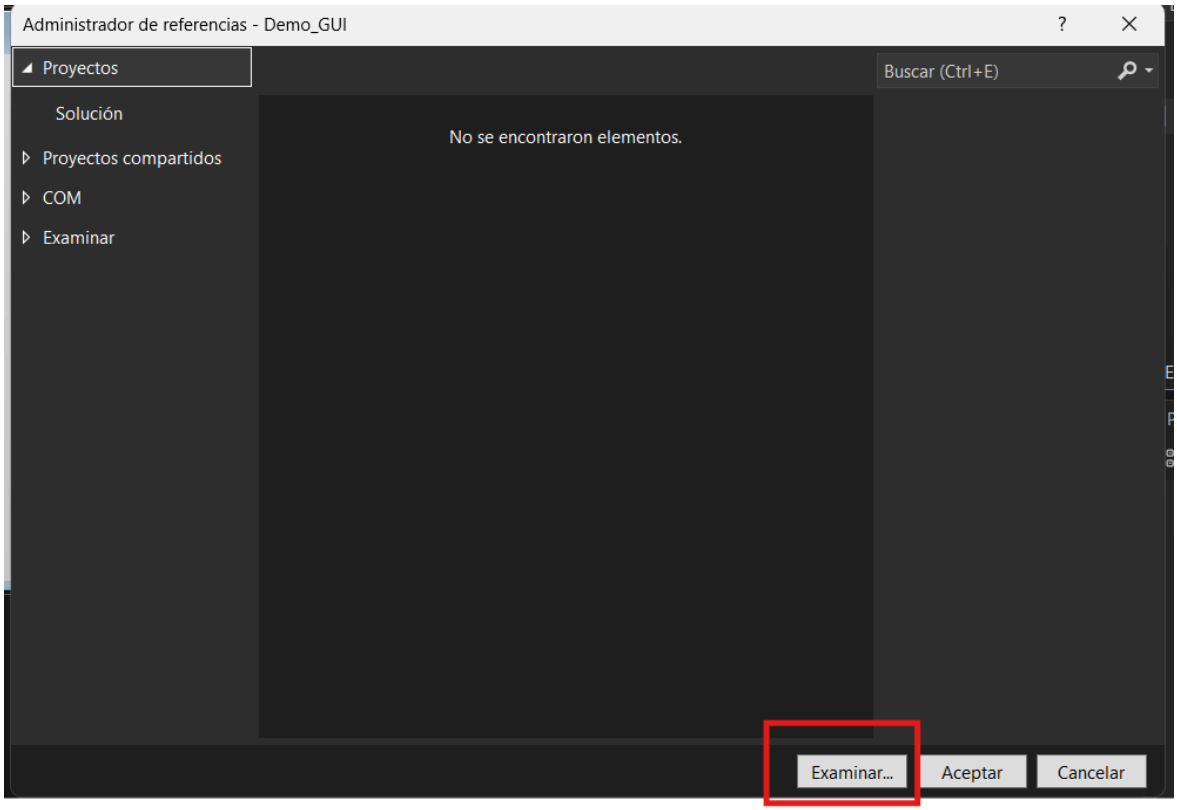
Paso 3: Vinculación de la Referencia Engine2D.dll

1. Dirígete al **Explorador de soluciones** a la derecha de la interfaz. Ubica el nombre del proyecto.
2. Despliega tu proyecto, haz clic derecho sobre la carpeta **Dependencias** y selecciona **Agregar referencia del proyecto...**.

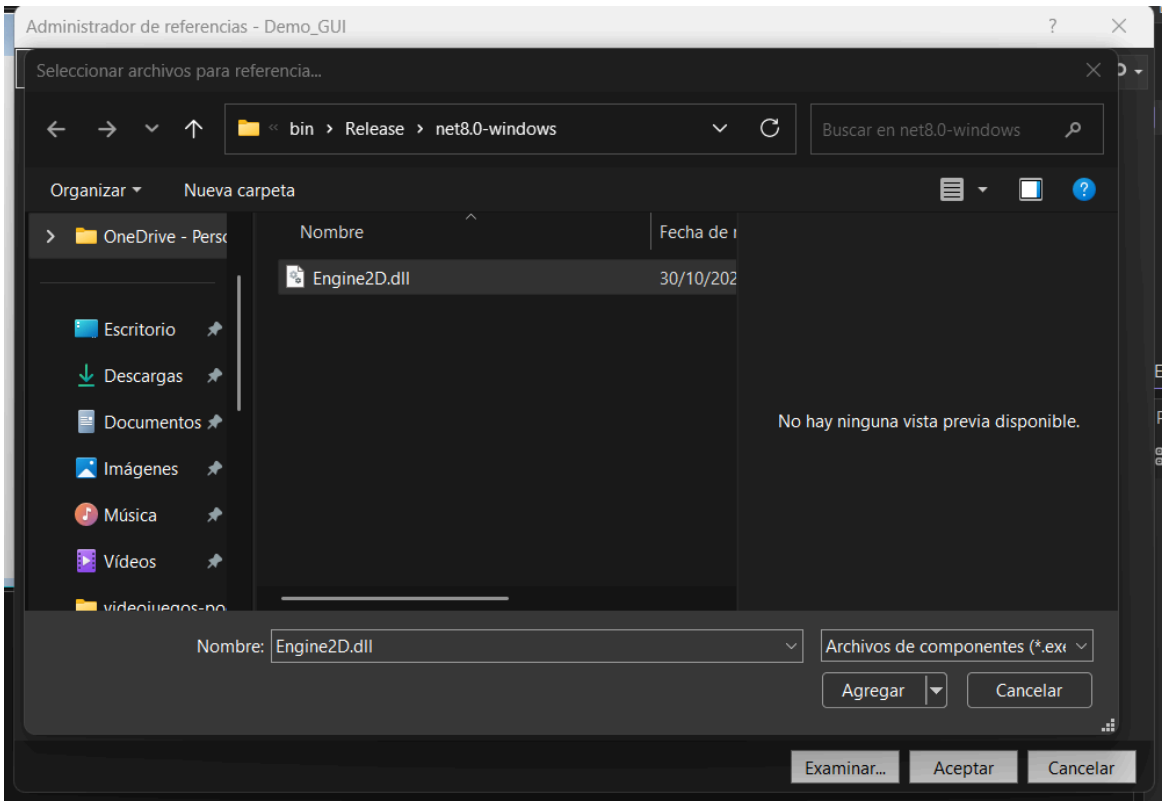




3. En la ventana del *Administrador de referencias*, haz clic en el botón **Examinar...** en la parte inferior.



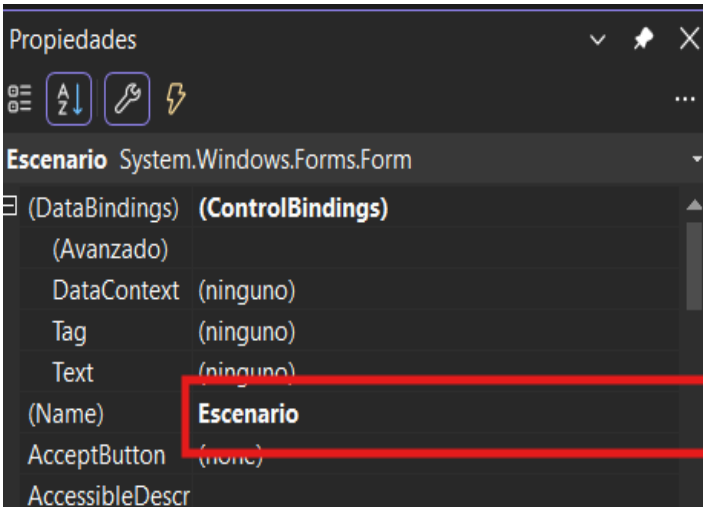
4. Navega hasta la ubicación donde descargaste Engine2D.dll, selecciónala y haz clic en **Agregar**.

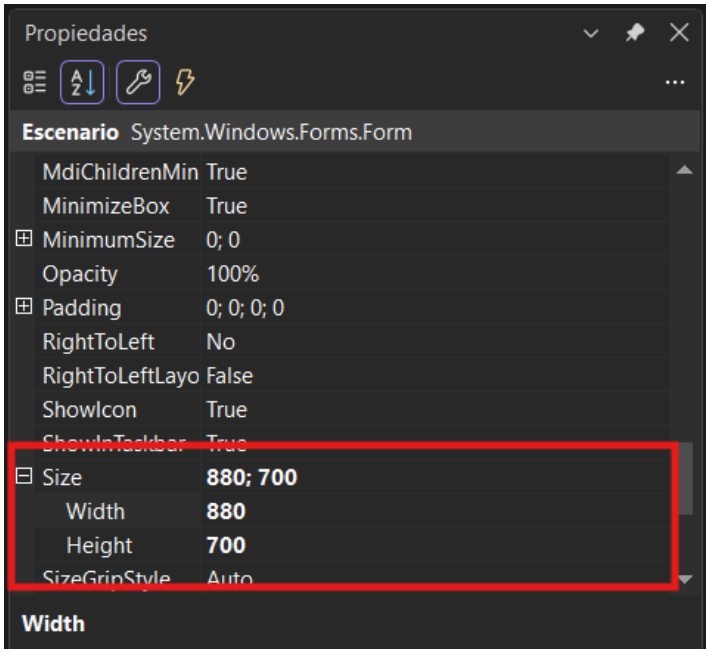


5. Verifica que quede marcada en la lista y presiona **Aceptar**.

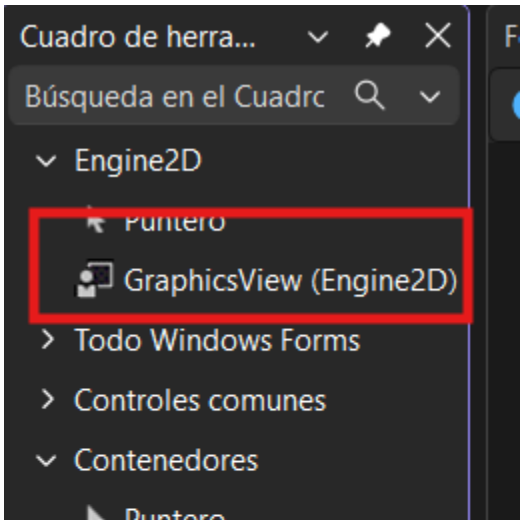
Paso 4: Configuración del Formulario y el Control GraphicsView

1. Abre el diseñador de Form1.cs. Recuerda que por sugerencia se recomienda cambiar los nombres por defecto que asigna el diseñador a los controles. En la ventana de **Propiedades**, cambia el nombre del formulario (Name) por ejemplo a Escenario y ajusta sus dimensiones (Size) a **880 × 700**.

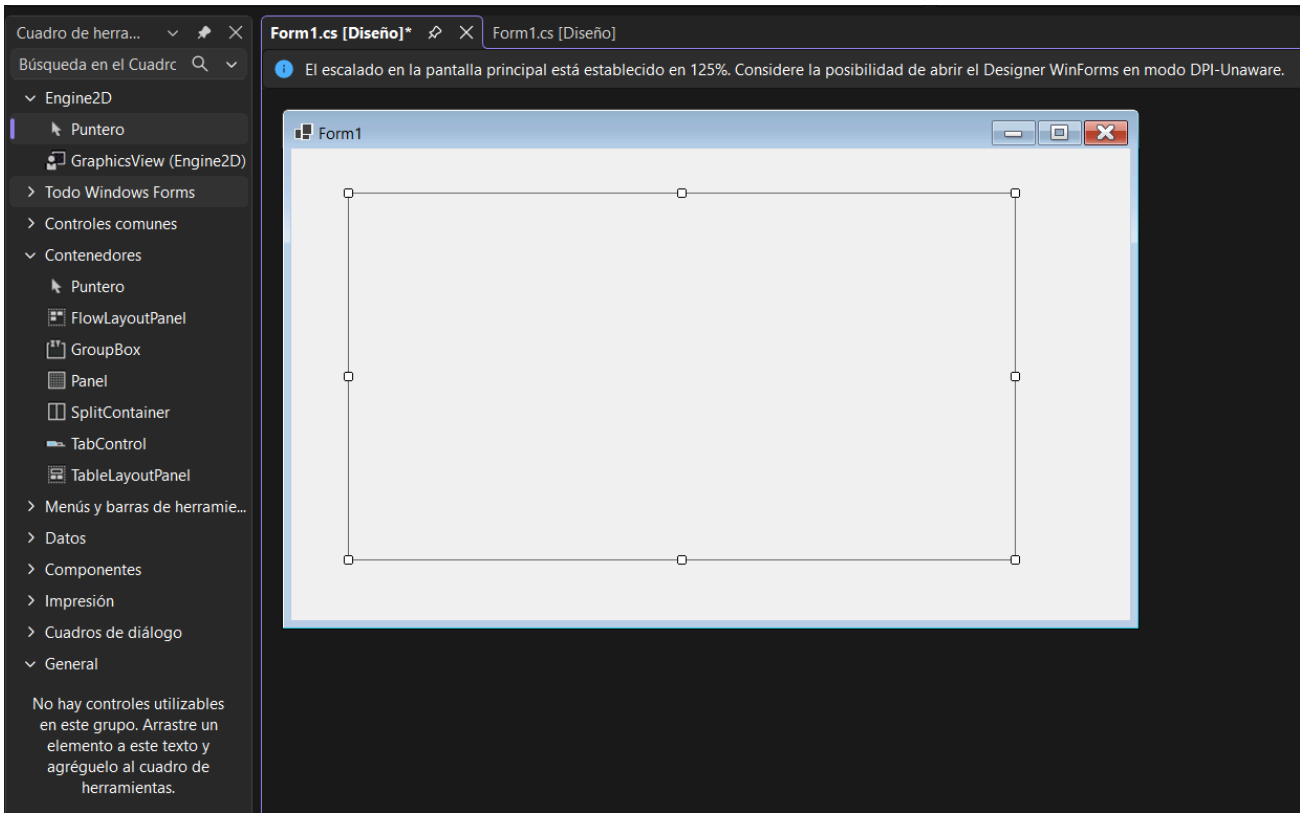


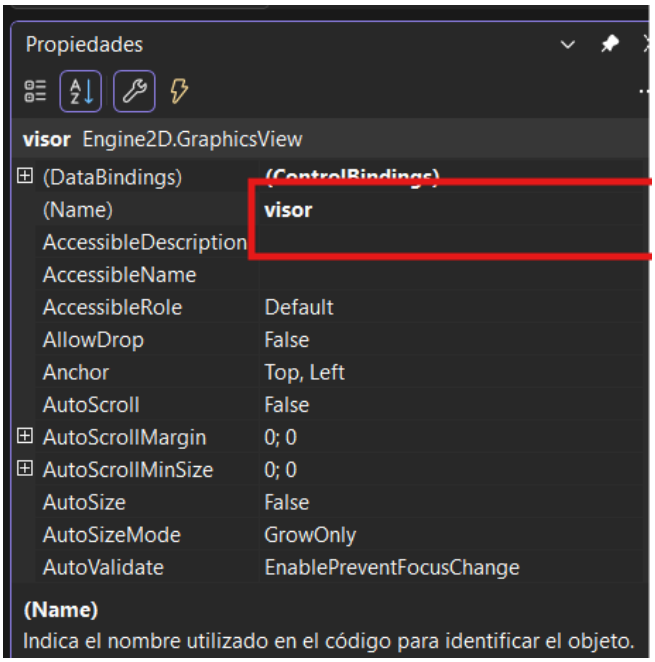


2. Abre el **Cuadro de herramientas**. Notarás que se ha creado la sección **Engine2D** con el componente **GraphicsView**. Este componente es parte de la librería que importaste y solo lo verás en este proyecto por haberlo referenciado si deseas usarlo en otro proyecto deberás hacer el mismo procedimiento del paso 3.



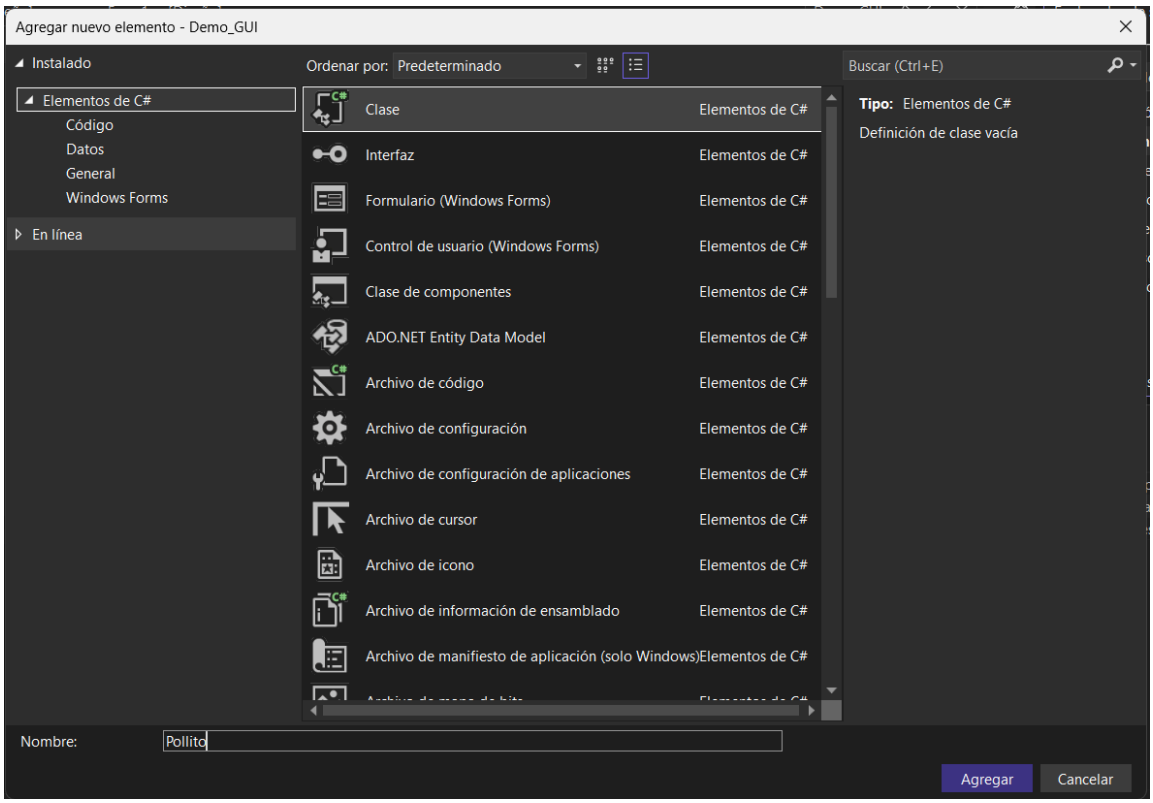
3. Arrastra un **GraphicsView** sobre el formulario como lo harías con otros controles por ejemplo con un botón verás en el formulario un recuadro que corresponde al componente GraphicsView por recomendación cambiale el nombre este componente, en la ventana de Propiedades asigna su nombre `(Name)` como por ejemplo **`visor`**.





Paso 5: Creación del Personaje

1. En el Explorador de soluciones, haz clic derecho sobre el proyecto Demo_GUI → **Agregar** → **Clase…**
2. Nombra el archivo con el nombre de la clase por ejemplo: **Pollito.cs** y haz clic en **Agregar**

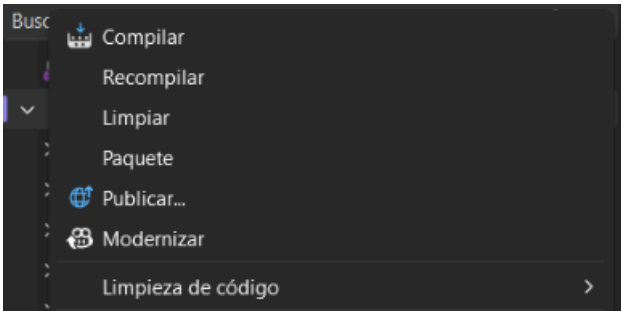


3. Hereda de la clase GraphicsItem, asegúrate de incluir la directriz using Engine2D

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Engine2D;

namespace Demo_GUI
{
    0 referencias
    internal class Pollito : GraphicsItem
    {
    }
}
```

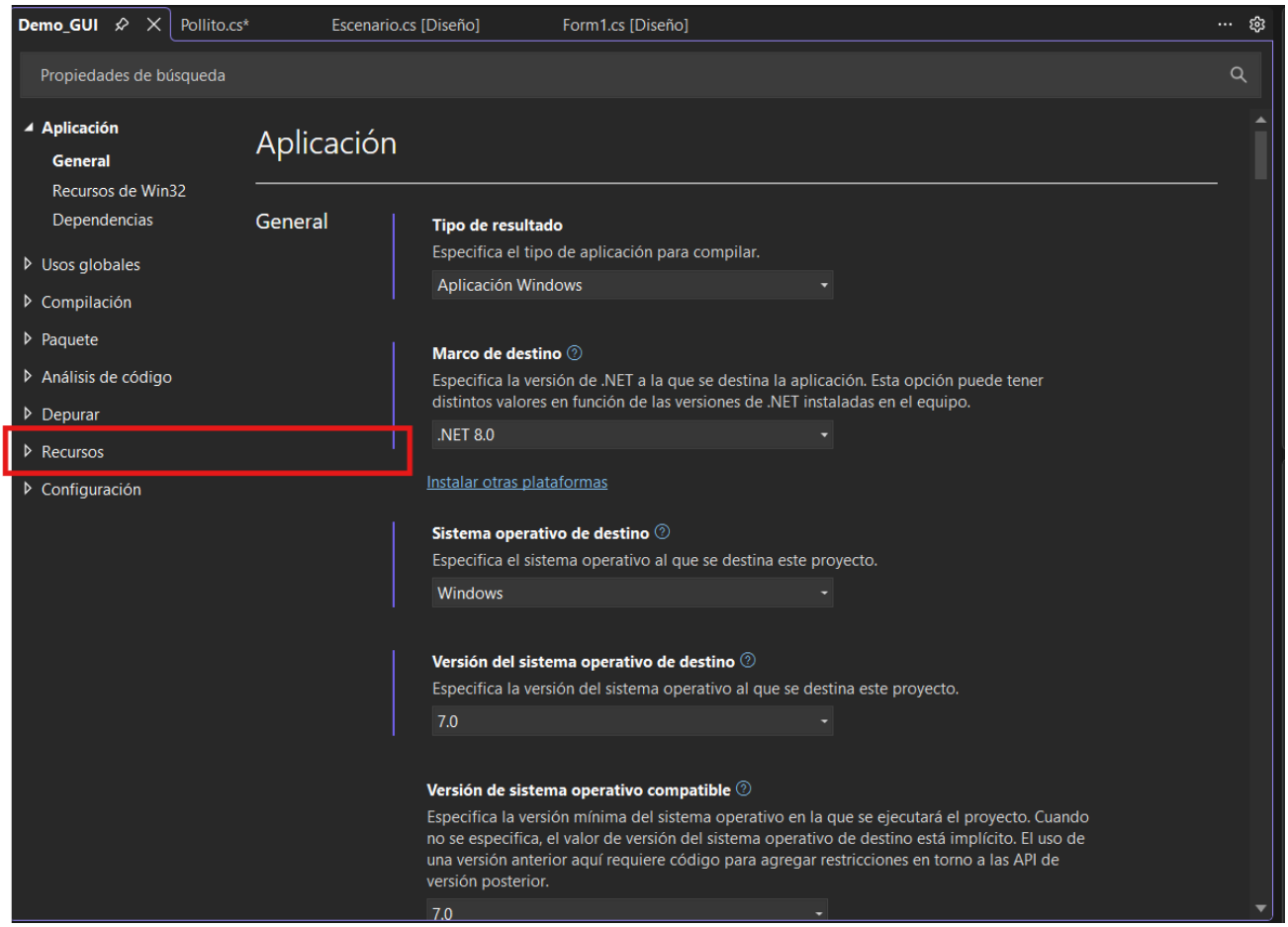
Paso 6: Gestión de Recursos Gráficos (.resx)

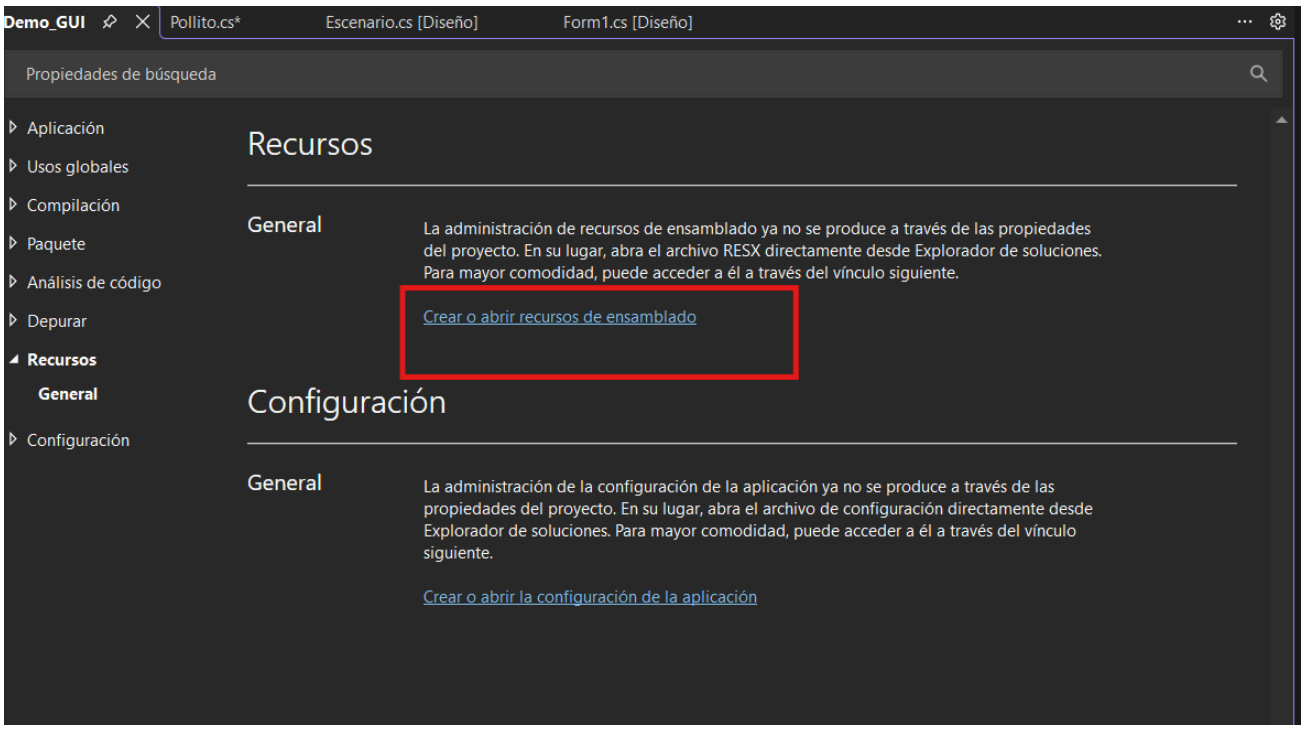


1. Haz clic derecho sobre el proyecto Demo_GUI y en menu que te aparece casi hasta el final de la lista seleccionas **Propiedades**.

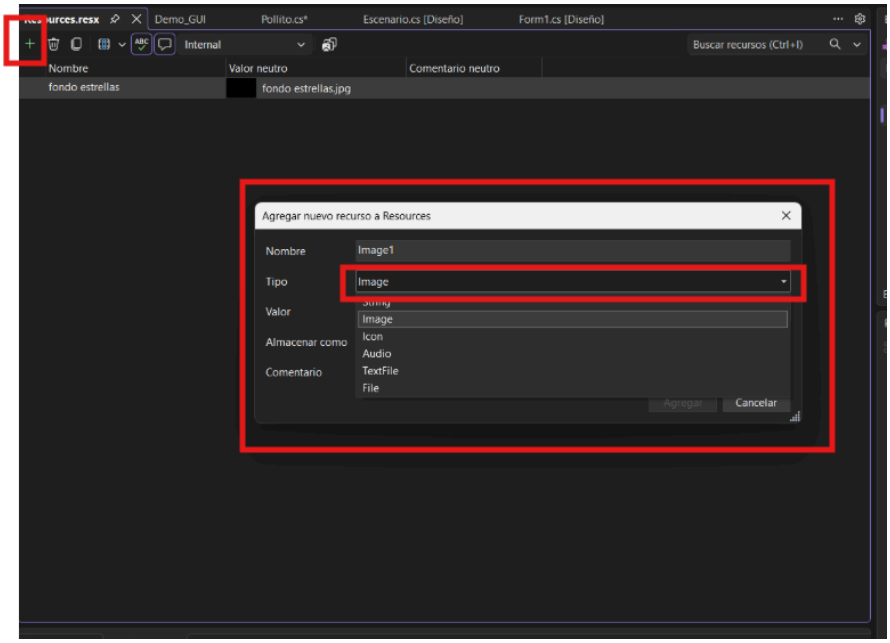
Nota: Este paso es necesario para incurrir gráficos usando al proyecto, de esta manera se moverán los recursos de su ubicación original a la del proyecto.De esta manera, las imágenes se empaquetan dentro del propio proyecto mediante rutas relativas. Esto garantiza que, si envías tu código a otro equipo o lo entregas en la plataforma académica, el juego seguirá encontrando los recursos gráficos sin errores de rutas faltantes.

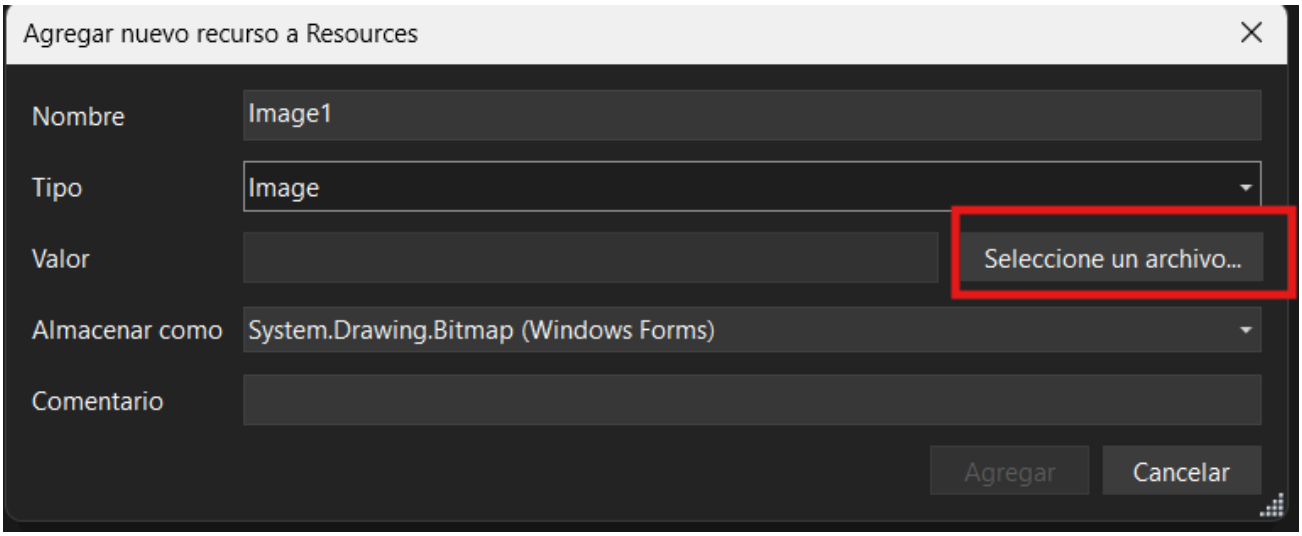
2. En la pestaña **Recursos**, haz clic en el enlace para **Crear o abrir recursos de ensamblado** (abre el archivo Resources.resx).


Demo_GUI
Pollito.cs*
Escenario.cs [Diseño]
Form1.cs [Diseño]
Propiedades de búsqueda
Aplicación
General
Recursos de Win32
Dependencias
Usos globales
Compilación
Paquete
Análisis de código
Depurar
Recursos
Configuración
Aplicación
General
Tipo de resultado
Especifica el tipo de aplicación para compilar.
Aplicación Windows
Marco de destino
Especifica la versión de .NET a la que se destina la aplicación. Esta opción puede tener distintos valores en función de las versiones de .NET instaladas en el equipo.
.NET 8.0
Instalar otras plataformas
Sistema operativo de destino
Especifica el sistema operativo al que se destina este proyecto.
Windows
Versión del sistema operativo de destino
Especifica la versión del sistema operativo al que se destina este proyecto.
7.0
Versión de sistema operativo compatible
Especifica la versión mínima del sistema operativo en la que se ejecutará el proyecto. Cuando no se especifica, el valor de versión del sistema operativo de destino está implícito. El uso de una versión anterior aquí requiere código para agregar restricciones en torno a las API de versión posterior.
7.0

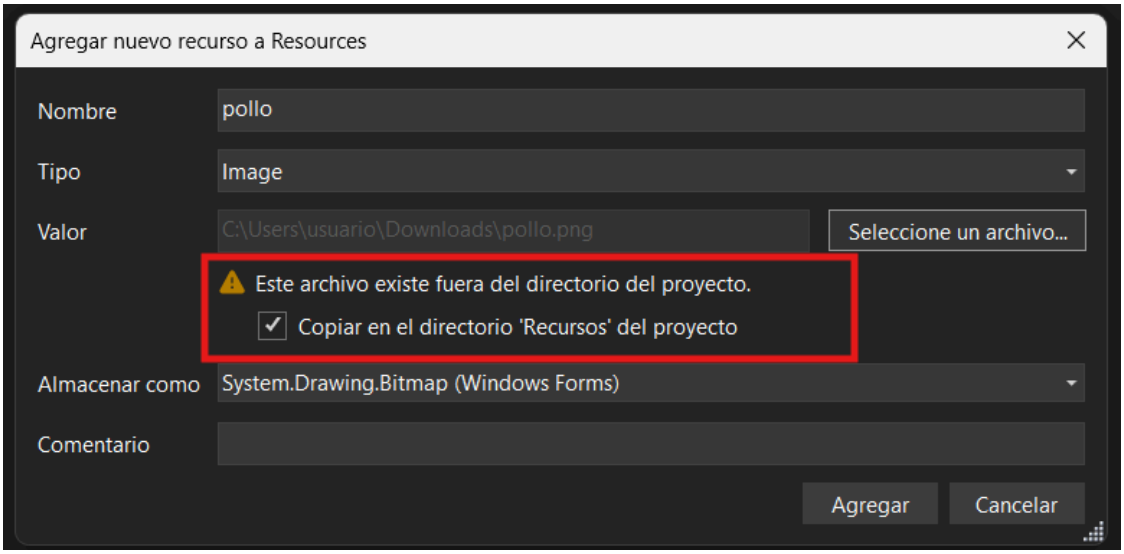

Demo_GUI
Pollito.cs*
Escenario.cs [Diseño]
Form1.cs [Diseño]
Propiedades de búsqueda
Aplicación
Usos globales
Compilación
Paquete
Análisis de código
Depurar
Recursos
General
Configuración
Recursos
General
La administración de recursos de ensamblado ya no se produce a través de las propiedades del proyecto. En su lugar, abra el archivo RESX directamente desde Explorador de soluciones. Para mayor comodidad, puede acceder a él a través del vínculo siguiente.
Crear o abrir recursos de ensamblado
Configuración
General
La administración de la configuración de la aplicación ya no se produce a través de las propiedades del proyecto. En su lugar, abra el archivo de configuración directamente desde Explorador de soluciones. Para mayor comodidad, puede acceder a él a través del vínculo siguiente.
Crear o abrir la configuración de la aplicación

3. Haz clic en la flecha del botón **Agregar recurso** (esquina superior izquierda) en la ventana emergente, en la opción **Tipo** seleccionas **Image** y en la opción **Valor** deberás dar clic en la opción selecciona un archivo y navegar hasta la carpeta donde se encuentran las imágenes.

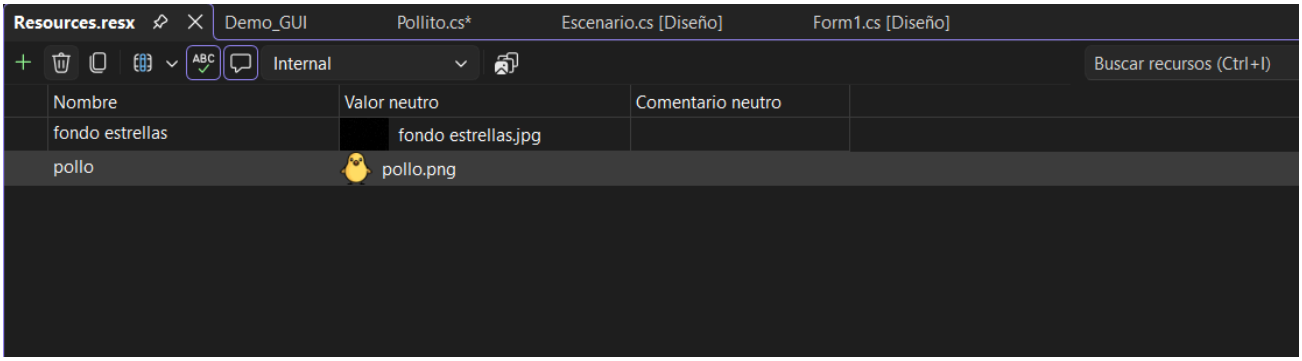


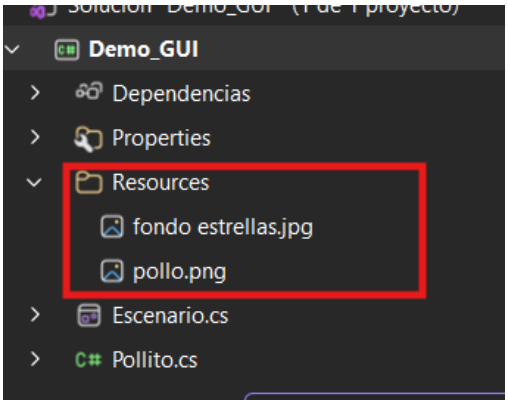


4. Selecciona las imágenes de tu proyecto. Si aparece la advertencia de archivo externo, marca la casilla **Copiar en el directorio 'Recursos' del proyecto** y confirma.



En la carpeta Resources deberán aparecer todas las imágenes que vayas agregando. tanto si abres la carpeta así como el árbol del proyecto, es decir desde el explorador de la solución.





Ya con los recursos gráficos incluidos en el proyecto continuaremos creando la clase **Pollito.cs**

Esta clase podrá incorporar los atributos de estado específicos y métodos propios de su comportamiento; sin embargo, al heredar de **GraphicsItem**, se le dotará automáticamente de las propiedades visuales necesarias para su representación.

A la clase base se le deben suministrar las coordenadas X e Y de ubicación del elemento, sus dimensiones de ancho (width) y alto (height), además del identificador del recurso gráfico sin extensión, asegurándote de haberlo incorporado previamente al proyecto.

el código nos queda así:

```
C#
using System;
using Engine2D;

namespace Demo_GUI
{
    internal class Pollito : GraphicsItem
    {
        // Atributos de estado
        private float x;
        private float y;
        private float vel;

        public Pollito(float x, float y, float vel)
            : base(x, y, 50, 50, "pollo") // Carga la imagen asignada a los recursos
"pollo"
        {
            this.x = x;
            this.y = y;
            this.vel = vel;
        }
    }
}
```

Paso 7: Inicialización de la Escena y del Objeto en Escenario.cs

1. Abre el archivo de código **Escenario.cs** e instancia la escena lógica
2. Crea una instancia del elemento **GraphicsScene**, declarándolo como un atributo para garantizar su visibilidad en toda la clase.

3. Posteriormente, establece la configuración general de la escena definiendo el origen de coordenadas mediante la propiedad **Origin**, especificando el punto inicial de referencia gráfica: **TopLeft** (esquina superior izquierda), **BottomLeft** (esquina inferior izquierda) o **Center** (centro de la pantalla). Para establecer un color de fondo, asigna un valor a la propiedad **BackgroundColor** (por ejemplo, *scene.BackgroundColor = Color.FromArgb(30, 30, 30)*; para lograr un fondo oscuro). Si prefieres agregar una imagen para el fondo, emplea la función **SetBackgroundImage** indicando el recurso asignado*: scene.SetBackgroundImage("fondo_estrellas")*;
4. Vincular Escena al Visor y definir el tamaño de la escena visor.Scene = scene; visor.Size = new Size(800, 600);
5. Agregamos el personaje pollito y lo incluiremos en la escena.
6. Para finalizar, debemos inicializar la escena mediante la sintaxis *this.Shown += (s, e) => visor.Start();*, lo cual permitirá activar el temporizador interno encargado de refrescar de manera constante la representación gráfica.

```
public partial class Escenario : Form
{
    5 referencias
    private GraphicsScene scene;
    2 referencias
    private Pollito pollito;

    1 referencia
    public Escenario()
    {

        InitializeComponent();

        StartPosition = FormStartPosition.CenterScreen;
        scene = new GraphicsScene();
        scene.Origin = CoordinateOrigin.TopLeft;
        scene.BackgroundColor = Color.FromArgb(30, 30, 30); // Fondo oscu
        visor.Scene = scene;
        visor.Size = new Size(800, 600);
        pollito = new Pollito(80, 80, 5);
        scene.AddItem(pollito);
        this.Shown += (s, e) => visor.Start();
    }
```

El código nos queda asi:

```C#
using System;
using System.Drawing;
using System.Windows.Forms;
using Engine2D;

namespace Demo_GUI
{
    public partial class Escenario : Form
    {
```

```
        private GraphicsScene scene;
        private Pollito pollito;

        public Escenario()
        {
            InitializeComponent();
            StartPosition = FormStartPosition.CenterScreen;

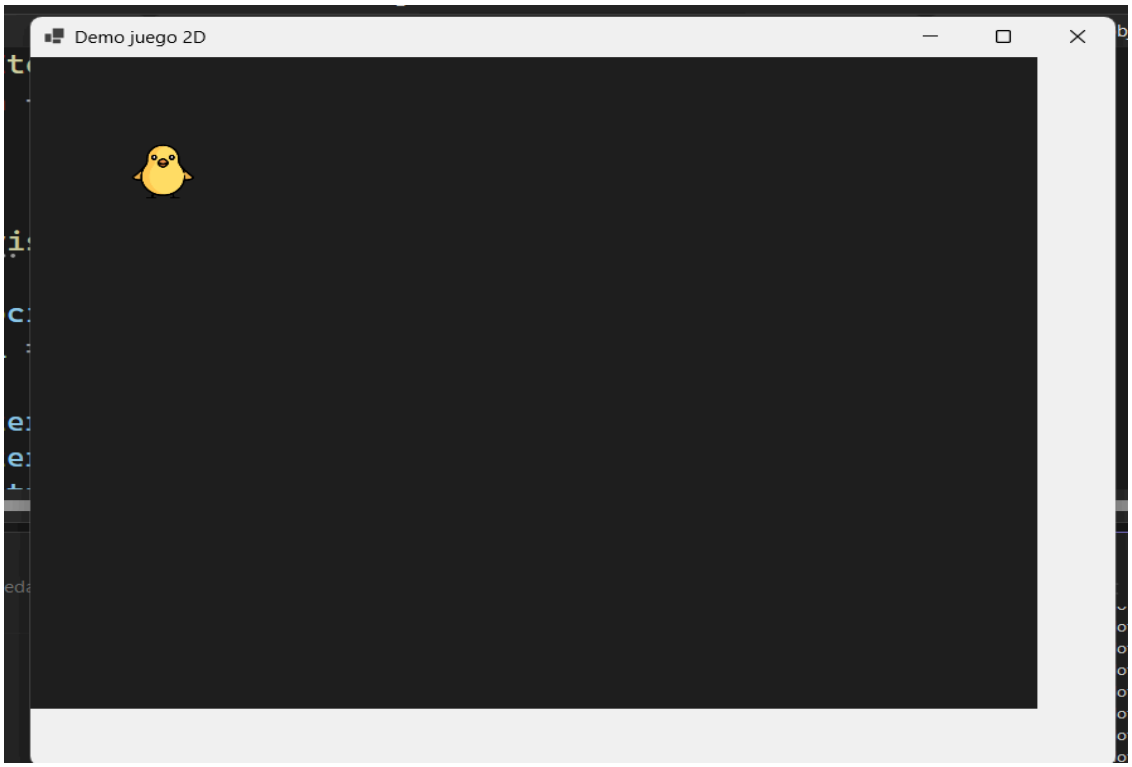
            // 1. Configurar Escena
            scene = new GraphicsScene();
            scene.Origin = CoordinateOrigin.TopLeft;
            scene.BackgroundColor = Color.FromArgb(30, 30, 30); // Fondo oscuro

            // 2. Vincular Escena al Visor
            visor.Scene = scene;
            visor.Size = new Size(800, 600);

            // 3. Crear e incluir al Personaje
            pollito = new Pollito(80, 80, 5);
            scene.AddItem(pollito);

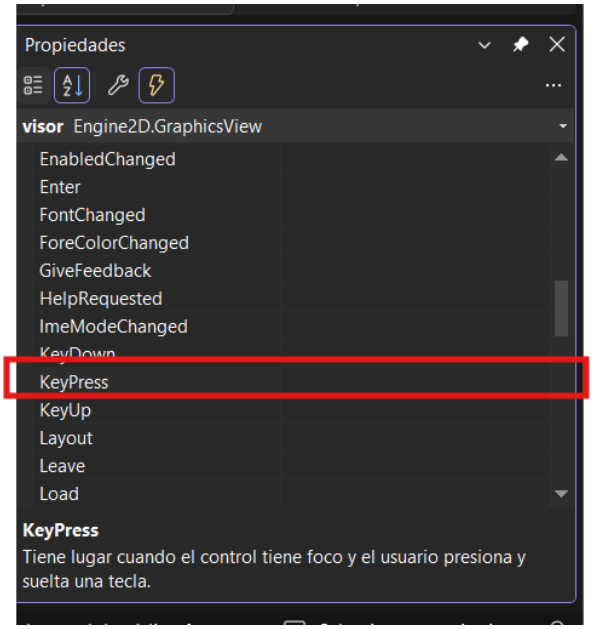
            // 4. Iniciar el bucle de renderizado al mostrar el formulario
            this.Shown += (s, e) => visor.Start();
        }
    }
}
```

Si ejecutamos el código deberíamos ver la siguiente escena



Paso 8: Manejo de Eventos del Teclado (Evento KeyPress)

1. En el Diseñador del Formulario, selecciona el control visor y ve a la pestaña de **Eventos** (ícono de rayo).
2. Haz doble clic sobre el evento **KeyPress** para generar su método en el código.



3. Escribe la lógica para modificar las coordenadas y la rotación del personaje

```C#
private void visor_KeyPress(object sender, KeyPressEventArgs e)
{
    float velocidad = 10f;
    char letra = char.ToUpper(e.KeyChar);

    switch (letra)
    {
        case 'A': // Mover Izquierda
            pollito.X -= velocidad;
            break;
        case 'D': // Mover Derecha
            pollito.X += velocidad;
            break;
        case 'W': // Mover Arriba
            pollito.Y -= velocidad;
            break;
        case 'S': // Mover Abajo
            pollito.Y += velocidad;
            break;
        case 'Q': // Rotar Izquierda
            pollito.Rotacion -= 5f;
            break;
        case 'E': // Rotar Derecha
            pollito.Rotacion += 5f;
            break;
    }
}
```

Al ejecutar nuevamente el proyecto, se desplegará la escena descrita. Al interactuar con las teclas configuradas, observarás cómo el personaje se desplaza fluidamente en las cuatro direcciones (arriba, abajo, izquierda y derecha). Adicionalmente, comprobarás que es posible modificar su rotación tanto en sentido horario como antihorario.

Paso 9: Creación de Muros y Manejo de Colisiones

1. Crearemos una clase denominada **Bloque.cs**. Esta estructura permitirá definir obstáculos geométricos dentro del entorno. El pollito me mueve libremente sin restricciones a lo largo del mapa, exceptuando las zonas donde detecte dichos elementos, bloqueando el paso a través de ellos.

2. De forma análoga a la implementación realizada con el personaje pollito, heredaremos de la clase base GraphicsItem para proveer a la entidad de capacidad de representación visual. En esta oportunidad no asignaremos un recurso de imagen; el componente se dibujará de forma predeterminada como un rectángulo ajustado a las dimensiones especificadas en el constructor (ancho y alto), por lo que únicamente se definirá la tonalidad mediante la propiedad **ColorItem**.

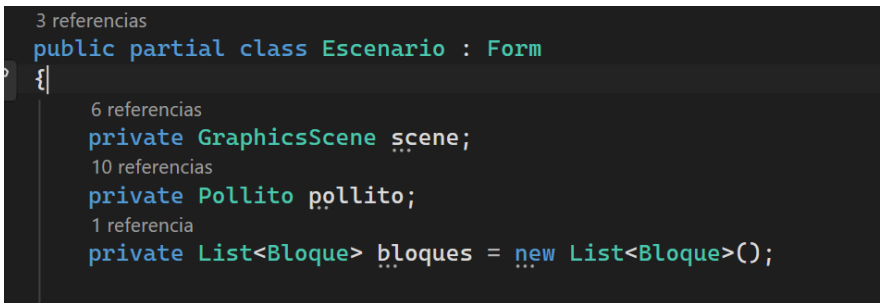
Nota: Es posible modificar la apariencia geométrica predeterminada asignando un valor a la propiedad Forma. De este modo, la entidad se representará como un círculo empleando la sintaxis *this.Forma = ItemShape.Circle;*.

El código nos queda de la siguiente manera:

```C#
using Engine2D;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Demo_GUI
{
    internal class Bloque : GraphicsItem
    {
        public Bloque(float x, float y, float ancho, float alto)
            : base(x, y, ancho, alto)
        {
            this.ColorItem = Color.OrangeRed;
        }
    }
}
```

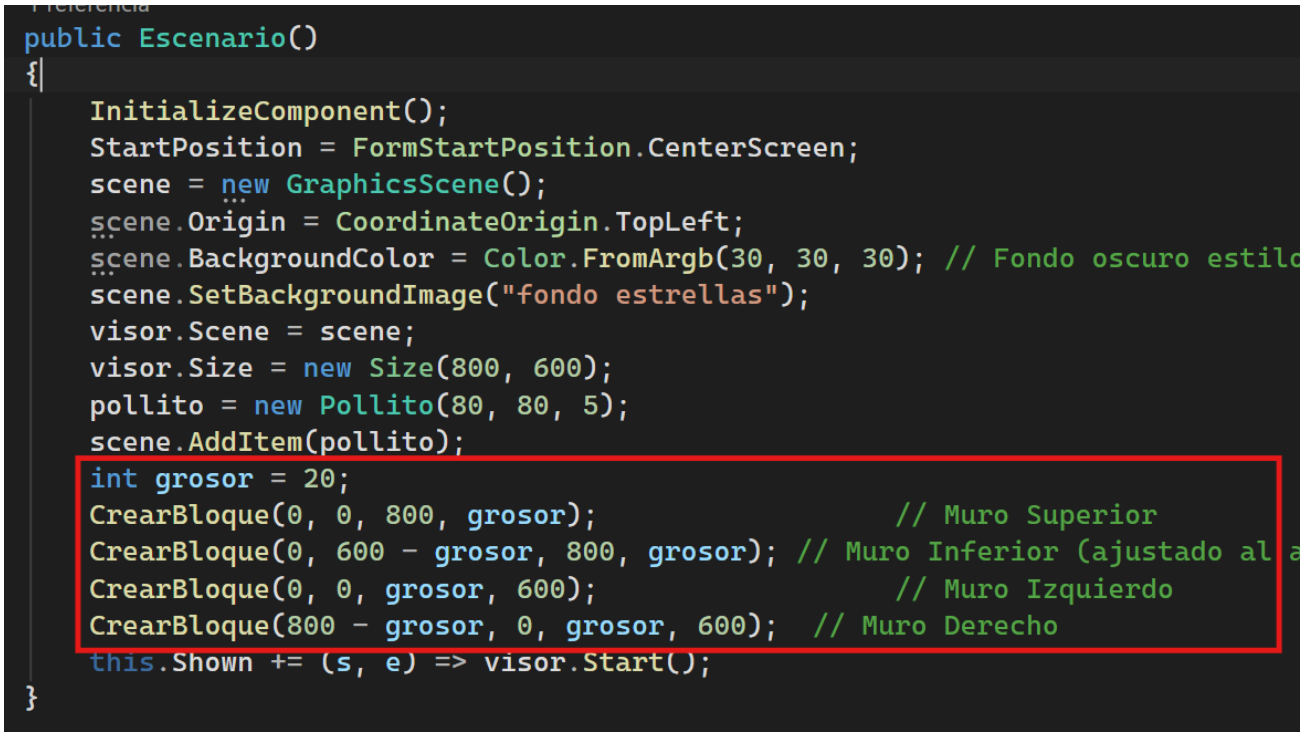
3. Ahora agregaremos los obstáculos al Escenario.cs. Incluimos un nuevo atributo a la clase Escenario. Una lista de objetos de tipo bloque.



4. Crearemos un método auxiliar para instanciar bloques.

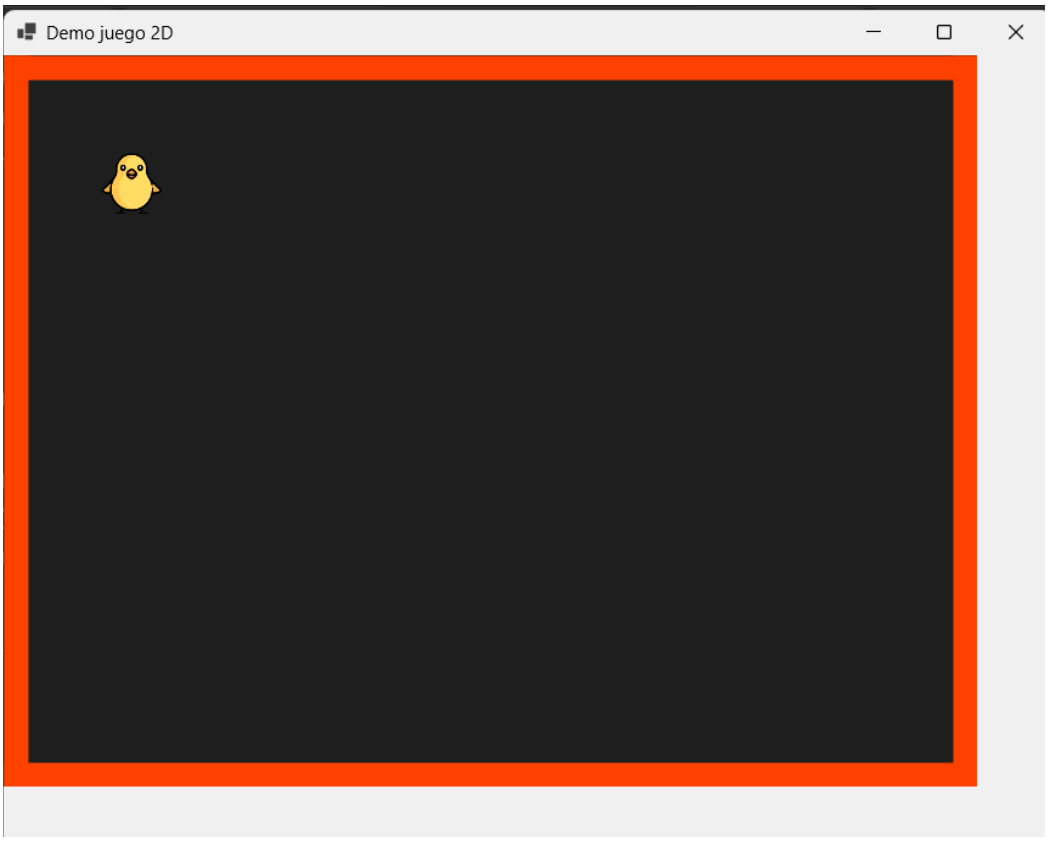
```C#
   void CrearBloque(float x, float y, float ancho, float alto)
    {
        Bloque pared = new Bloque(x, y, ancho, alto);
        bloques.Add(pared); // Guardamos la referencia para colisiones
        scene.AddItem(pared);   // Agregamos a la escena para renderizar
    }
```

5. Mediante el metodo **CrearBloque**, se instanciamos cuatro muros perimetrales (superior, inferior, lateral izquierda y lateral derecha) que delimitar los contornos de la escena.



```
public Escenario()
{
    InitializeComponent();
    StartPosition = FormStartPosition.CenterScreen;
    scene = new GraphicsScene();
    scene.Origin = CoordinateOrigin.TopLeft;
    scene.BackgroundColor = Color.FromArgb(30, 30, 30); // Fondo oscuro estilo
    scene.SetBackgroundImage("fondo estrellas");
    visor.Scene = scene;
    visor.Size = new Size(800, 600);
    pollito = new Pollito(80, 80, 5);
    scene.AddItem(pollito);
    int grosor = 20;
    CrearBloque(0, 0, 800, grosor);                  // Muro Superior
    CrearBloque(0, 600 - grosor, 800, grosor); // Muro Inferior (ajustado al a
    CrearBloque(0, 0, grosor, 600);                  // Muro Izquierdo
    CrearBloque(800 - grosor, 0, grosor, 600);  // Muro Derecho
    this.Shown += (s, e) => visor.Start();
}
```

Al ejecutar el código veremos el pollito encerrado en los 4 muros de color naranja.



5. Para concluir, adaptaremos la lógica dentro del controlador del evento de teclado con la finalidad de restringir el paso del personaje a través de los muros. Registramos la posición previa del objeto antes de procesar el movimiento especificado por la tecla. Tras efectuar el desplazamiento, se verificará la interacción física con cada obstáculo almacenado en la lista; si se detecta una colisión con cualquier muro, se restaurarán las coordenadas registradas previamente.

El código ajustado para el método quedará de la siguiente manera.

```C#
private void visor_KeyPress(object sender, KeyPressEventArgs e)
{
    float velocidad = 10f;
    char letra = char.ToUpper(e.KeyChar);

    float xAnterior = pollito.X;
    float yAnterior = pollito.Y;
    switch (letra)
    {
        case 'A':
            pollito.X -= velocidad;
            break;
        case 'D':
            pollito.X += velocidad;
            break;
        case 'W':
```

```
            pollito.Y -= velocidad;
            break;

        case 'S':
            pollito.Y += velocidad;
            break;

        case 'Q': // Rotar a la izquierda
            pollito.Rotacion -= 5f;
            break;

        case 'E': // Rotar a la derecha
            pollito.Rotacion += 5f;
            break;
    }
    if (pollito.ColisionaCon(bloques))
    {
        pollito.SetPos(xAnterior, yAnterior);

    }
}
```

Al ejecutar este código podras comprobar como el pollito se mueve en todas la dirección según se le indique con las teclas pero con la restricción de no poder atravesar los muros.